

Programming and Data Structures - II (CS3201)

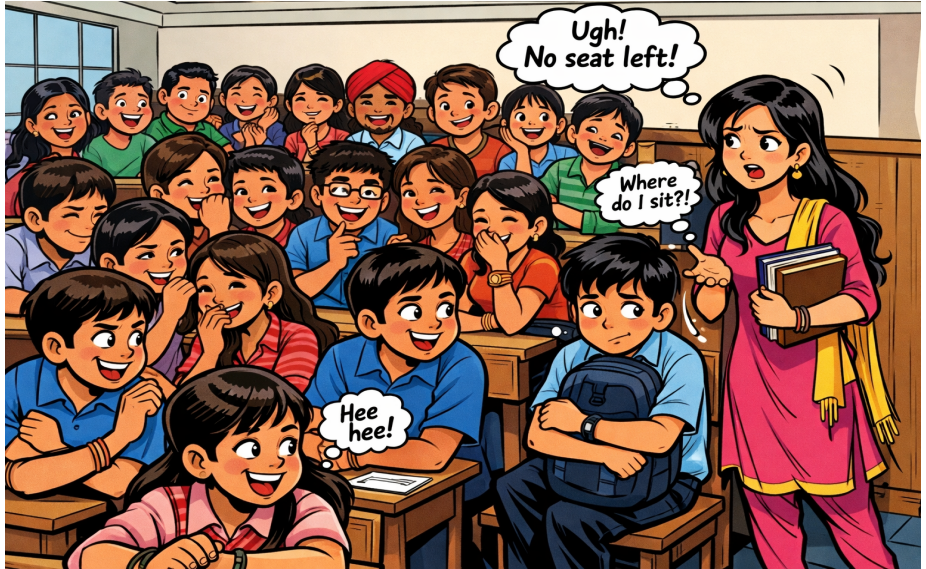
Lecture 2

Kripabandhu Ghosh

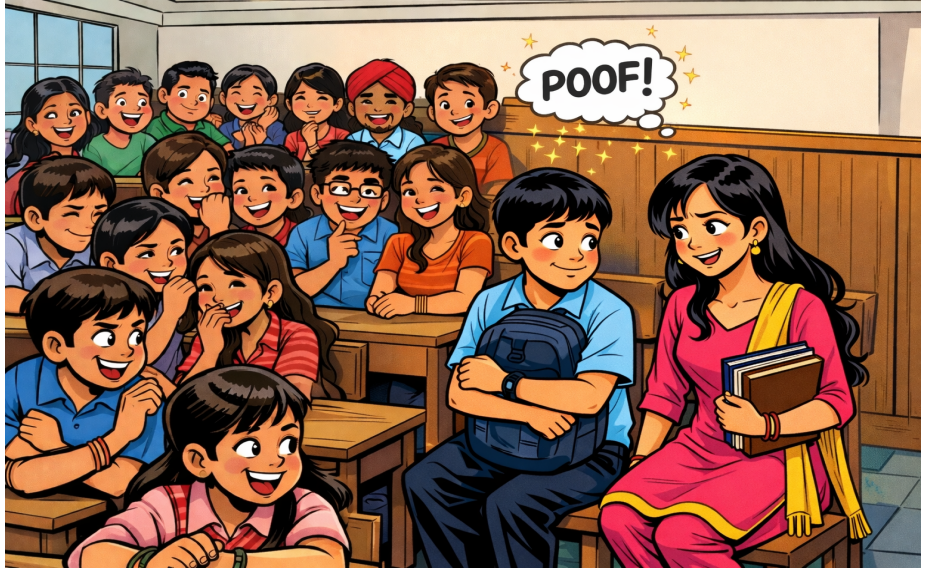
CDS, IISER Kolkata

LINKED LISTS

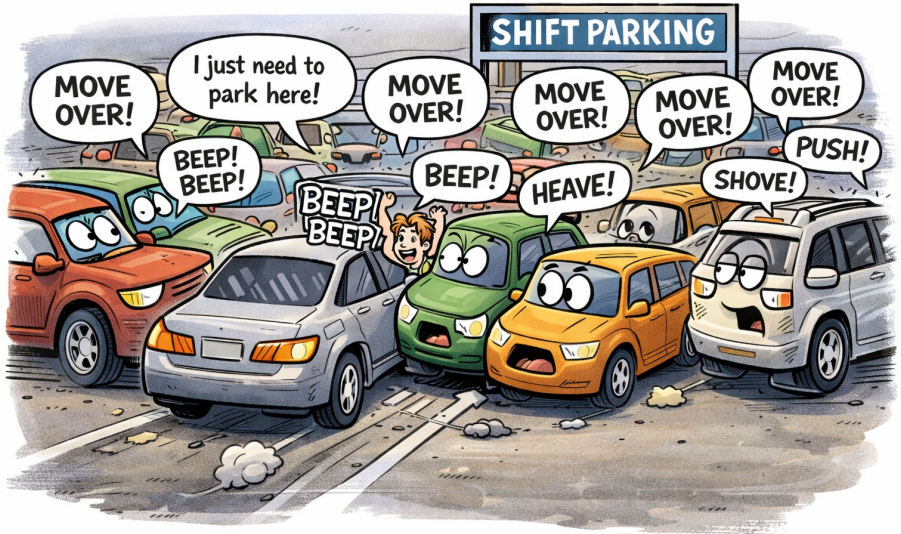
Story of static arrays: No seats left!



Story of dynamism: We have a new seat!

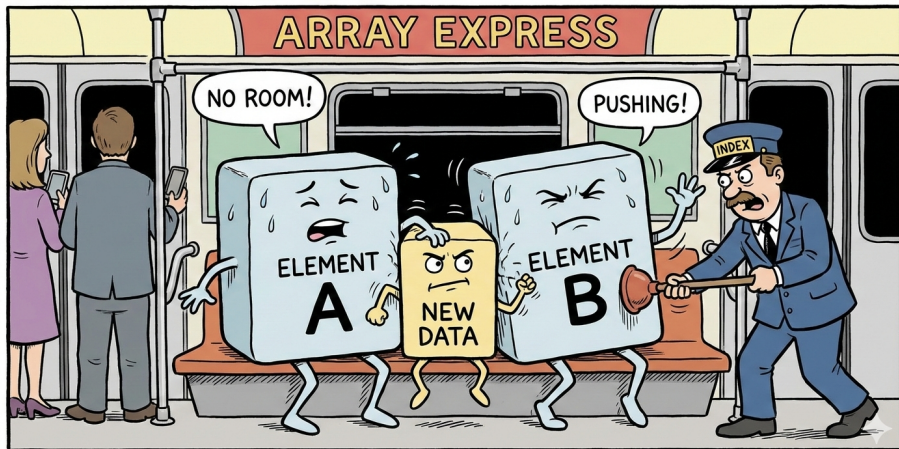


Insertion Problem in Arrays (Even if there are extra spaces)



Insertion Problem in Arrays

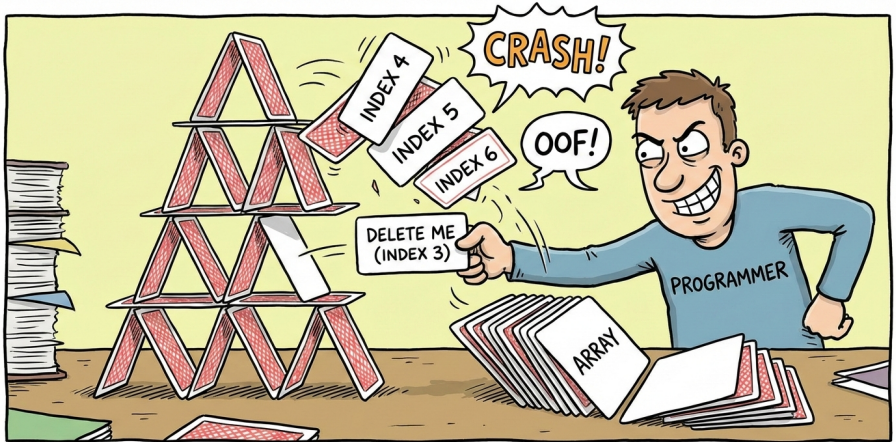
THE INSERTION STRUGGLE



DELETING FROM AN ARRAY...



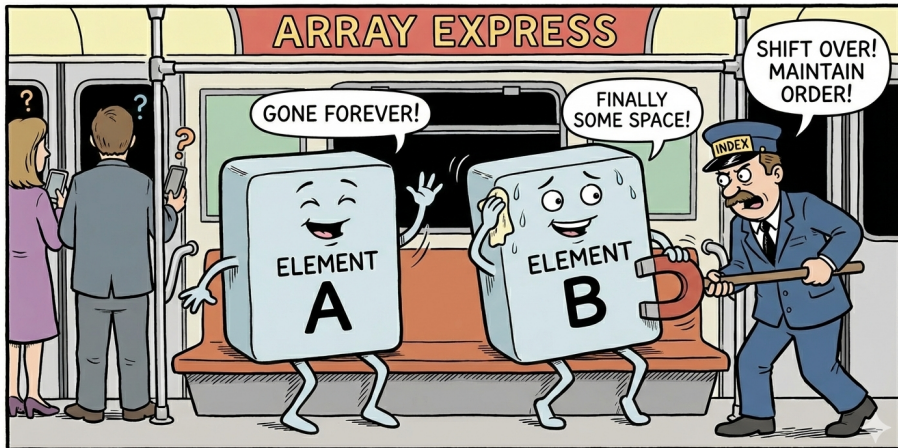
Deletion Problem in Arrays



THE ARRAY DELETION PROBLEM!

Deletion Problem in Arrays

THE DELETION DILEMMA.



Linked lists

Features

- Dynamic data structure (array is a static data structure, size to be specified precisely at the beginning)
 - Size can grow or shrink throughout the program
- No wastage of memory space – insertion when space is required, deletion when space is no longer required
- Insertion and deletion efficient – no moving of data

Disadvantages

- Linear search to access a data item (array can be accessed directly)
- More storage than an array of the same number of elements (data and pointer)

Why not realloc()?

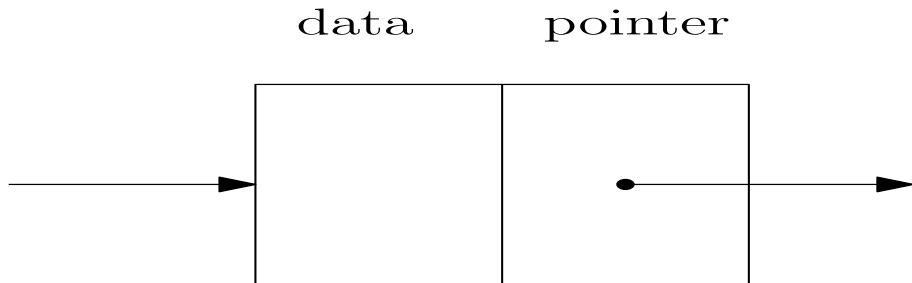
- realloc() needs *contiguous space* and might need to copy the entire array to a new location
- After many alloc/free cycles, may fail to find space even with enough total memory (Memory fragmentation) and can have memory leaks if not well handled $\Rightarrow$ **unreliable for real-life dynamic systems**
- The issues of insertion/deletion towards the beginning or at the middle remain

Linked lists: node

Format

- **Data:** information stored
- **Pointer:** address of the next node

Linked lists: node



Node

Linked lists: node (implementation)

Code

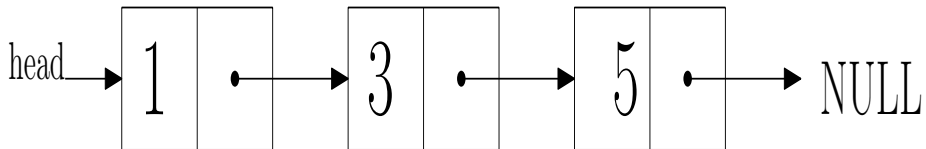
```
struct node
{
    int info;
    struct node *link;
};
```

Linked lists

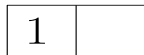
Organization

- **head/start**: Pointer to the start of the linked list
- The first node is pointed at by **head/start**
- Each node (except the last one) points to the next node
- The last node points to NULL

Linked lists



Linked lists: insertion at the beginning (visualization)



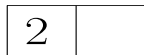
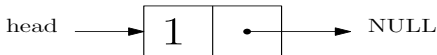
(to be inserted)

INSERTION
AT THE BEGINNING

(a) Empty list:



(b) Insertion into an empty list



(to be inserted)

(c) Insertion again



Linked lists: insertion at the beginning (implementation)

Code

```
struct node *addatbeg(struct node *start,int data)
{
    struct node *tmp;
    tmp = (struct node *)malloc(sizeof(struct node));
    tmp->info = data;
    tmp->link = start;
    start = tmp;
    return start;
}
```

Linked lists: traversal

Code

```
struct *p
p = start;
while(p != NULL)
{
    printf( "%d ",p->info);
    p = p->link;
}
```

Linked lists: insertion at the end (visualization)



(to be inserted)

INSERTION
AT THE END

Before insertion



After insertion



Linked lists: insertion at the end (implementation)

Code

```
struct node *addatend(struct node *start,int data)
{
    struct node *p,*tmp;
    tmp = (struct node *)malloc(sizeof(struct node));
    tmp->info = data;
    p = start;
    while(p->link != NULL)
        p = p->link;
    p->link = tmp;
    tmp->link = NULL;
    return start;
}
```

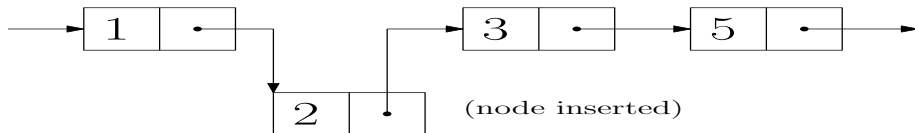
Linked lists: insertion after (visualization)

BEFORE INSERTION

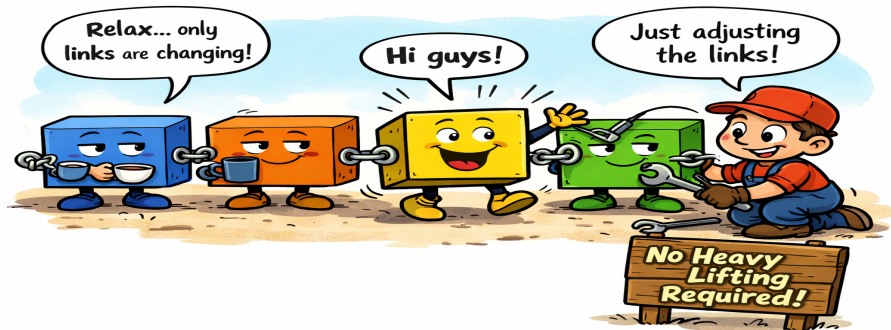


(node to be inserted)

AFTER INSERTION



Linked lists: insertion after (comic relief)

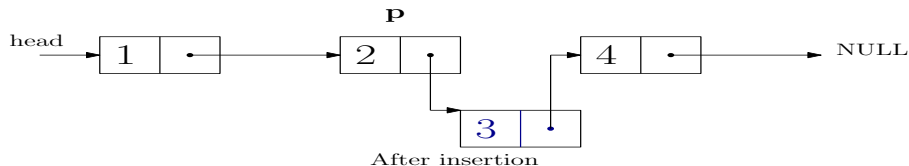
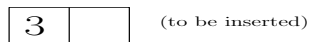
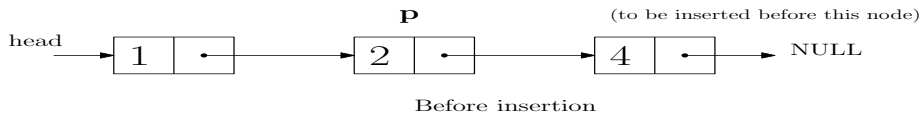


Linked lists: insertion after (implementation)

Code

```
struct node *addafter(struct node *start, int data, int item) //To insert 'data' after 'item'
{
    struct node *tmp,*p;
    p=start;
    while(p!=NULL)
    {
        if(p->info==item)
        {
            tmp=(struct node *)malloc(sizeof(struct node));
            tmp->info=data;
            tmp->link=p->link;
            p->link=tmp;
            return start;
        }
        p=p->link;
    }
    printf( "%d not present in the list\n",item);
    return start;
}
```

Linked lists: insertion before (visualization)



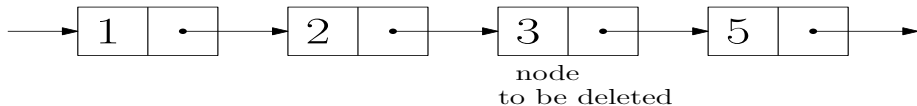
Linked lists: insertion before (implementation)

Code

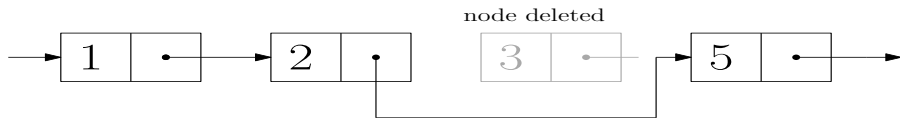
```
struct node *addbefore(struct node *start,int data,int item) //To insert 'data' before 'item'
{
    struct node *tmp,*p;
    if(start == NULL )
    {
        printf("List is empty\n");
        return start;
    }
    /*If data to be inserted before first node*/
    if(item == start->info)
    {
        tmp = (struct node *)malloc(sizeof(struct node));
        tmp->info = data;
        tmp->link = start;
        start = tmp;
        return start;
    }
    p=start;
    while(p->link != NULL)
    {
        if(p->link->info == item)
        {
            tmp = (struct node *)malloc(sizeof(struct node));
            tmp->info = data;
            tmp->link = p->link;
            p->link = tmp;
            return start;
        }
        p = p->link;
    }
    printf("%d not present in the list\n",item);
    return start;
}
```

Linked lists: deletion (visualization)

BEFORE DELETION



AFTER DELETION

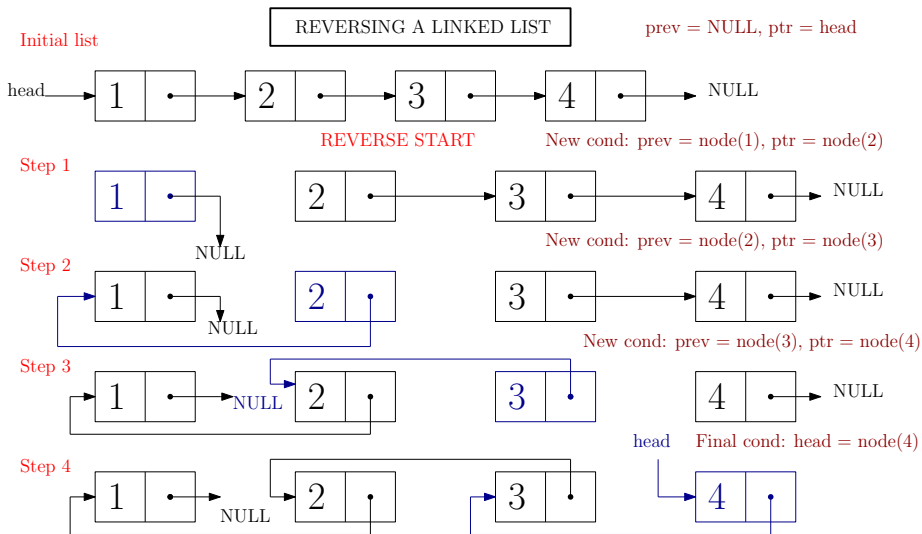


Linked lists: deletion (implementation)

Code

```
struct node *del(struct node *start,int data)
{
    struct node *tmp,*p;
    if(start==NULL)
    {
        printf("List is empty\n");
        return start;
    }
    /*Deletion of first node*/
    if(start->info==data)
    {
        tmp=start;
        start=start->link;
        free(tmp);
        return start;
    }
    /*Deletion in between or at the end*/
    p=start;
    while(p->link!=NULL)
    {
        if(p->link->info==data)
        {
            tmp=p->link;
            p->link=tmp->link;
            free(tmp);
            return start;
        }
        p=p->link;
    }
    printf("Element %d not found\n",data);
    return start;
}
```

Linked lists: reversing (visualization)



Linked lists: reversing (implementation)

Code

```
struct node *reverse(struct node *start)
{
    struct node *prev, *ptr, *next;
    prev = NULL;
    ptr = start;
    while(ptr != NULL)
    {
        next = ptr->link;
        ptr->link = prev;
        prev = ptr;
        ptr = next;
    }
    start = prev;
    return start;
}
```

Disadvantages of a (singly) linked list

Disadvantages

- Traversal in one direction, no provision to go to the previous node
- Not possible to *delete* a current node or insert *before* a current node

Disadvantages of a (singly) linked list

Disadvantages

- Traversal in one direction, no provision to go to the previous node
- Not possible to *delete* a current node or insert *before* a current node

Solution

Doubly linked list

One-way traffic!



Deletion of the current node: Might be useful!

